



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA  
CURSO: BACHARELADO EM ENGENHARIA DE CONTROLE E  
AUTOMAÇÃO

# Detecção de quedas em Tempo Real para Monitoramento de Idosos Utilizando Redes Neurais Embarcadas

Discentes: Carlos Augusto Leal Memoria, Felipe Carvalho da Silva, Miguel Ângelo de Lacerda Silva e Renan Saraiva dos Santos.

Orientador: Prof. Dr. Raphael Maciel de Sousa.

**Cajazeiras - PB**

**Agosto de 2025**

## Resumo

Este trabalho propõe o desenvolvimento de um algoritmo baseado em redes neurais profundas para a classificação em tempo real de sinais de vibração obtidos por acelerômetros de smartphones. O objetivo é monitorar o *status* de indivíduos, especialmente idosos, identificando padrões de movimento como caminhar, sentar-se e cair. Os dados foram coletados diretamente da plataforma *Edge Impulse*, a partir de simulações experimentais, e processados para treinamento e validação do modelo. A rede neural projetada alcançou 87,6% de acurácia, com *F1-score* médio de 0,88 no conjunto de validação. Os resultados demonstram a viabilidade da abordagem para aplicações em monitoramento assistivo e prevenção de quedas, com potencial para evolução mediante ajustes das arquiteturas da rede neural e maior robustez do conjunto de dados.

## 1. Introdução

O envelhecimento populacional é um fenômeno global que impõe novos desafios aos sistemas de saúde e à sociedade. Nesse contexto, a segurança e o bem-estar dos idosos tornam-se uma prioridade, especialmente no que tange à prevenção e detecção de quedas, que representam uma das principais causas de morbidade e mortalidade nessa faixa etária (World Health Organization, 2021). A tecnologia, por meio de sistemas de *Ambient Assisted Living* (AAL), surge como uma aliada fundamental, oferecendo soluções para o monitoramento não invasivo e contínuo.

A ubiquidade dos smartphones, equipados com uma gama de sensores inerciais como acelerômetros e giroscópios, os posiciona como uma plataforma ideal para a implementação de sistemas de Reconhecimento de Atividades Humanas (HAR). Esses dispositivos combinam capacidade computacional, conectividade e baixo custo, eliminando a necessidade de hardware dedicado (Wang et al., 2019).

O aprendizado de máquina, e mais especificamente as redes neurais profundas (*Deep Learning*), revolucionou a capacidade de extrair padrões complexos de dados de séries temporais, como os sinais de vibração provenientes de acelerômetros. Modelos neurais podem aprender a distinguir assinaturas características de diferentes atividades, desde movimentos cotidianos, como andar e sentar, até eventos anômalos e de alto risco, como uma queda.

Este trabalho se insere neste contexto, propondo a criação de um classificador de atividades em tempo real, desenvolvido na plataforma *Edge Impulse*. O objetivo é validar a eficácia de uma rede neural densa (*DNN*) para classificar os estados 'andando', 'sentando' e 'caindo', a partir de dados coletados por um *smartphone*, visando uma futura aplicação em um sistema de alerta para cuidadores e familiares.

## 2. Justificativa

A pesquisa contribui para o campo do aprendizado de máquina embarcado (*Embedded ML*) ao avaliar a performance de uma arquitetura de rede neural densa em uma tarefa de classificação de alta complexidade com recursos computacionais limitados. A utilização da plataforma *Edge Impulse* como ferramenta de ponta-a-ponta (desde a coleta de dados até a validação do modelo) oferece um estudo de caso prático sobre a viabilidade e os desafios de democratizar o desenvolvimento de soluções de *IA* para dispositivos de borda.

A principal motivação prática é o desenvolvimento de uma tecnologia assistiva de baixo custo e alta acessibilidade. Um sistema de detecção de quedas baseado em smartphones pode oferecer uma camada de segurança vital para idosos que vivem sozinhos, permitindo uma resposta rápida em caso de emergência. A capacidade de classificar múltiplos estados (e não apenas quedas) enriquece o sistema, permitindo um monitoramento mais contextualizado do nível de atividade do usuário.

## 3. Objetivos

### 3.1. Objetivo Geral

Desenvolver e validar um modelo de rede neural embarcado, capaz de classificar em tempo real as atividades humanas de 'andar', 'sentar' e 'cair' a partir de dados de vibração de um acelerômetro de smartphone, visando a aplicação em sistemas de monitoramento de idosos.

### 3.2. Objetivos Específicos

- Coletar um conjunto de dados de sinais de acelerômetro correspondentes às três classes de atividades, utilizando um *smartphone* e a plataforma *Edge Impulse*.
- Realizar o pré-processamento dos dados, incluindo normalização e extração de características (*features*) a partir dos sinais brutos.
- Projetar, treinar e otimizar uma arquitetura de rede neural densa para a tarefa de classificação.
- Avaliar rigorosamente o desempenho do modelo utilizando métricas padrão, como acurácia, matriz de confusão, precisão, *recall* e *F1-score*.
- Analisar criticamente os resultados e propor aprimoramentos na arquitetura do modelo para trabalhos futuros.

## 4. Referencial teórico

#### **4.1. Envelhecimento populacional e quedas**

O envelhecimento populacional tem imposto desafios crescentes à saúde pública, especialmente no que diz respeito à prevenção e à detecção de quedas. De acordo com a Organização Mundial da Saúde (World Health Organization, 2021), as quedas estão entre as principais causas de morbidade e mortalidade em idosos, o que reforça a necessidade de soluções de monitoramento contínuo e não invasivo. Nesse contexto, os sistemas de *Ambient Assisted Living* (AAL) representam alternativas relevantes para garantir maior segurança e autonomia.

#### **4.2. Smartphones como plataforma de monitoramento**

A disseminação dos *smartphones*, equipados com sensores inerciais como acelerômetros e giroscópios, possibilitou sua aplicação em sistemas de Reconhecimento de Atividades Humanas (HAR – Human Activity Recognition). Esses dispositivos apresentam vantagens como conectividade, capacidade de processamento e baixo custo, eliminando a necessidade de hardware dedicado (WANG et al., 2019).

Estudos recentes reforçam esse cenário ao mostrar que sensores de *smartphones* podem fornecer dados confiáveis para monitoramento em tempo real e em ambientes não controlados (GRU FRAMEWORK, 2022; SENSOR-EMBEDDED CNNs, 2023)

#### **4.3. Aprendizado de máquina embarcado (Embedded ML)**

O aprendizado profundo (*Deep Learning*) tem revolucionado a análise de sinais temporais, como os provenientes de acelerômetros. Redes neurais profundas apresentam alta capacidade de identificar padrões complexos, possibilitando a detecção de diferentes atividades humanas, incluindo eventos críticos como quedas (WANG et al., 2019).

Pesquisas exploram diferentes arquiteturas, como Redes Neurais Recorrentes (RNNs) e suas variações (*LSTM*, *GRU*), que são eficazes na captura de dependências temporais dos dados (*RNN HAR FRAMEWORK*, 2021). Outras abordagens exploram mecanismos de atenção temporal e de canal para melhorar a robustez em cenários de variabilidade dos sinais (*GRU + ATTENTION*, 2022).

#### **4.4. Arquiteturas de redes neurais aplicadas**

O avanço do aprendizado de máquina embarcado (*Embedded Machine Learning – Embedded ML*) permitiu que algoritmos de inteligência artificial fossem executados em dispositivos de borda com recursos limitados. Plataformas como a *Edge Impulse* oferecem ferramentas de ponta a ponta para coleta, pré-processamento, treinamento e validação de modelos, democratizando o desenvolvimento de aplicações em monitoramento assistivo (CLASSIFICAÇÃO DE ATIVIDADES HUMANAS..., 2023).

Diante disso, arquiteturas mais avançadas vêm sendo investigadas:

- CNNs embarcadas em microcontroladores mostraram viabilidade para HAR em tempo real com baixo consumo (SENSOR-EMBEDDED CNNs, 2023).

- Modelos híbridos CNN-GRU exploram tanto características espaciais (CNN) quanto temporais (GRU), resultando em ganhos de robustez e precisão (HYBRID CNN-GRU, 2021).
- 3D-CNNs têm sido estudadas para análise contextualizada de movimentos em janelas temporais, com aplicação em monitoramento assistivo (3D-CNN HAR, 2023).

#### 4.5.1. Perceptron de Múltiplas Camadas (*MLP – Multi-Layer Perceptron*)

A arquitetura *MLP*, também conhecida como rede neural densa ou totalmente conectada (RIBEIRO, 2024), representa a fundação das redes neurais profundas (Equação 1). Neste modelo, cada neurônio de uma camada está conectado a todos os neurônios da camada subsequente, formando uma estrutura densa de interconexões.

$$z = w \cdot x + b = \sum_{i=1}^n (w_i \cdot x_i + b) \quad \text{Eq. (1)}$$

A operação de um único neurônio em uma camada densa é uma transformação linear seguida por uma função de ativação não-linear.

Para a camada de saída em um problema de classificação multiclasse, a função de ativação *Softmax* é empregada (Equação 2). Ela converte os valores de saída brutos (*logits*) de  $K$  neurônios em uma distribuição de probabilidade, onde cada saída representa a probabilidade de a entrada pertencer a uma das  $K$  classes.

$$P(y = j|x) = \text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K (e^{z_k})} \quad \text{Eq. (2)}$$

#### 4.5.2. Redes Neurais Convolucionais (*CNNs – Convolutional Neural Networks*)

Originalmente concebidas para o processamento de imagens, as *CNNs* foram adaptadas com grande sucesso para dados de séries temporais (*1D-CNNs*), segundo Aquino (2024). Sua principal vantagem reside na capacidade de aprender hierarquias de características locais diretamente dos dados brutos, preservando a informação temporal.

A operação central da *CNN* é a convolução. Um filtro (ou *kernel*) de tamanho  $m$  desliza sobre a série temporal de entrada, realizando um produto de ponto em cada posição (Equação 2). Isso permite que a rede detecte padrões específicos, como picos, vales ou oscilações, independentemente de onde eles ocorram na janela de tempo. A operação de convolução *1D* é definida como:

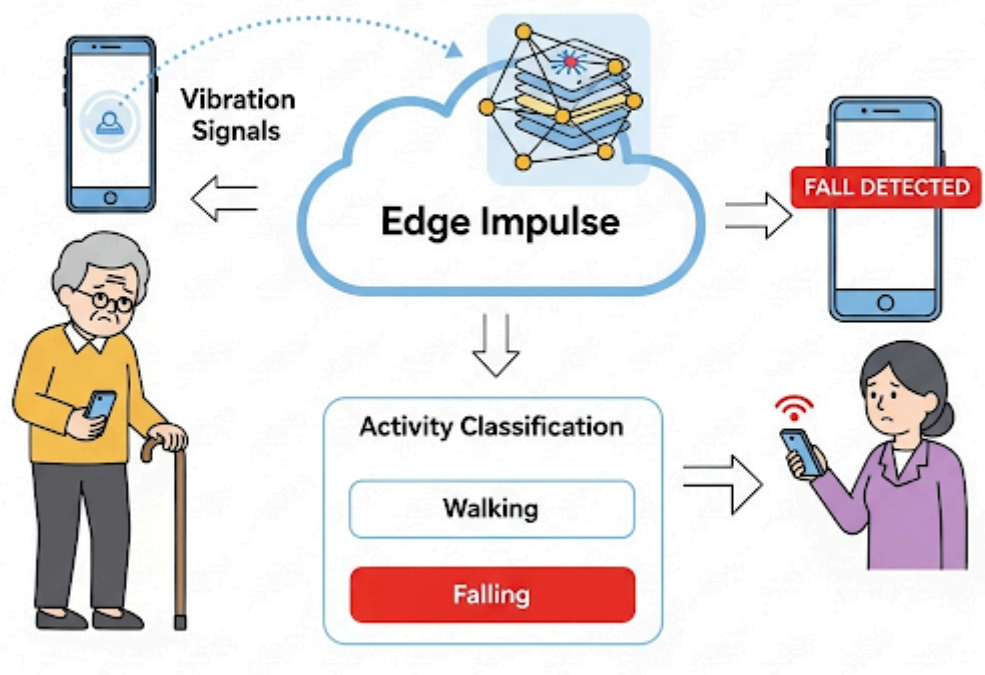
$$(s \cdot K)[t] = \sum_{i=1}^m (s[t - i] \cdot k[i]) \quad \text{Eq. (2)}$$

Onde  $(s \cdot k)[t]$  é a saída do filtro no passo de tempo  $t$ . A utilização de múltiplos filtros em uma camada permite que a rede aprenda a detectar diversos padrões simultaneamente. Após a convolução, é comum aplicar uma camada de *Pooling*, que reduz a dimensionalidade dos dados, mantendo as características mais salientes e conferindo um grau de invariância a pequenas translações no sinal.

## 5. Metodologia

A metodologia adotada neste trabalho abrange um fluxo de desenvolvimento completo, desde a aquisição e preparação dos dados até a implementação e validação de um modelo de aprendizado de máquina para a classificação de atividades em tempo real. A fase inicial consistiu na coleta de dados, utilizando um *smartphone* como sensor inercial para capturar os sinais de aceleração correspondentes a três classes de atividade pré-definidas: 'Andando', 'Sentando' e 'Caindo'. A aquisição e a rotulagem (anotação) das amostras foram gerenciadas pela plataforma de aprendizado de máquina embarcado *Edge Impulse*. Subsequentemente, todo o ciclo de vida do modelo – incluindo o pré-processamento do sinal, o projeto da arquitetura da rede neural, o treinamento e os testes – foi conduzido dentro do mesmo ecossistema. O modelo resultante foi então validado quanto à sua capacidade de realizar a inferência em tempo real, classificando novas amostras de vibração não vistas durante a fase de desenvolvimento. O fluxograma completo deste processo é ilustrado esquematicamente na Figura 1.

**Figura 1-** Ilustração do projeto: Sistema de Monitoramento de atividades de Idosos



**Fonte:** Autores (2025).

### 5.1. Coleta e Estruturação dos Dados

O conjunto de dados foi construído através da simulação das três atividades de interesse. Um *smartphone* foi utilizado para capturar os dados do acelerômetro nos eixos *accX*, *accY* e *accZ*. A coleta foi gerenciada pela plataforma *Edge Impulse*.

- Classes: andando, caindo, sentando.
- Amostras coletadas: Foram gravadas 36 amostras de 15 segundos cada, resultando em um dataset total de 9 minutos.
- Divisão do Dataset: As amostras foram divididas em dois conjuntos:
  - Treinamento: 28 amostras (andando: 12, caindo: 12, sentando: 4).
  - Teste: 8 amostras (andando: 3, caindo: 4, sentando: 1).

A plataforma *Edge Impulse* aplicou uma técnica de janelamento (*windowing*) sobre os dados de treinamento, gerando 1848 janelas de análise, o que aumentou significativamente o volume de dados para o treinamento efetivo da rede. O conjunto de janelas foi então dividido em 78% para treinamento e 22% para validação.

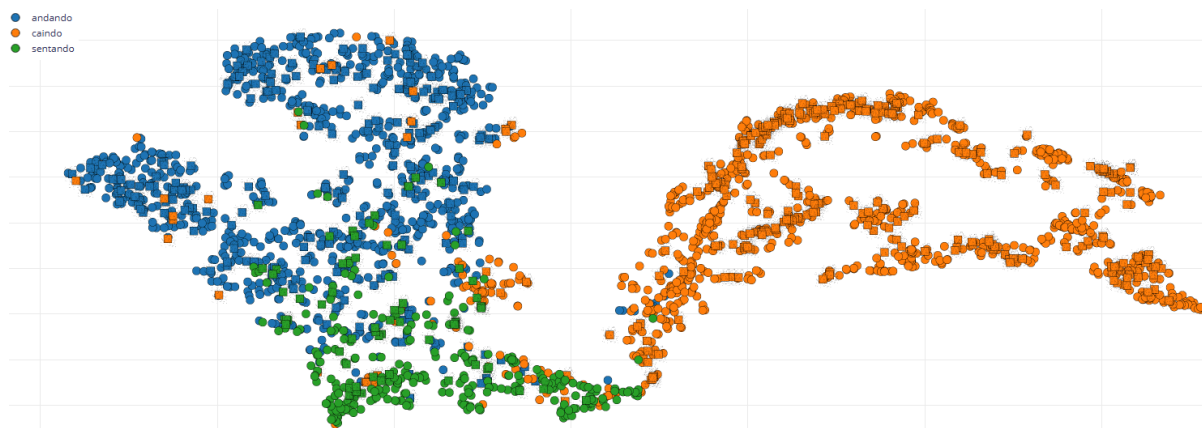
## 5.2. Pré-processamento e Extração de Características

Antes de alimentar a rede neural, os dados brutos de cada janela passaram por um bloco de processamento de sinal digital (DSP). Este bloco extraiu um vetor de 1247 características, combinando métricas no domínio do tempo (RMS, desvio padrão) e no domínio da frequência (picos de FFT, energia espectral). Para garantir a estabilidade do treinamento, as características foram normalizadas utilizando o *StandardScaler* da biblioteca *Scikit-learn*, que centraliza os dados na média 0 com desvio padrão 1.

Para a visualização da separabilidade dos dados, foram aplicadas técnicas de redução de dimensionalidade como *t-SNE* e *PCA*.

Para visualização dos dados foi usado o *t-SNE* (*t-Distributed Stochastic Neighbor Embedding*), um método não-linear que se destaca por preservar a estrutura local dos dados, sendo particularmente eficaz na identificação de agrupamentos (*clusters*). A formação de *clusters* visualmente distintos, especialmente na projeção *t-SNE*, como ilustrado na Figura 2.

**Figura 2 - t-SNE.**

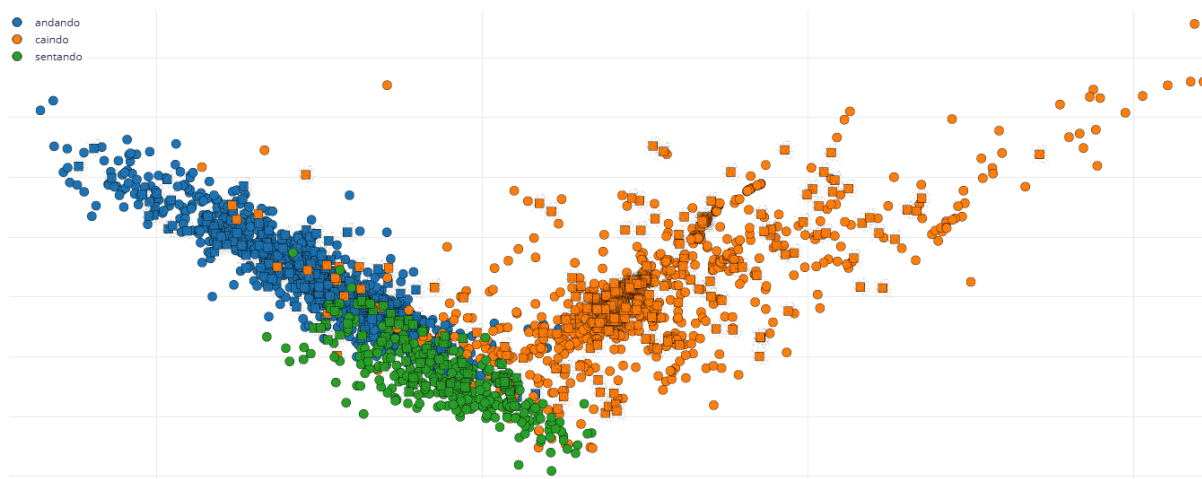


**Fonte:** Autores (2025).

A análise de Componentes Principais (*PCA*) é uma técnica de transformação linear que busca um novo sistema de coordenadas, conhecido como componentes principais, de tal forma que a maior variância dos dados seja capturada nos primeiros componentes. Assim, tal técnica foi utilizada para também fornecer um vislumbre mais detalhado dos dados que foram coletados, veja na Figura 3.

**Figura 3 - PCA.**



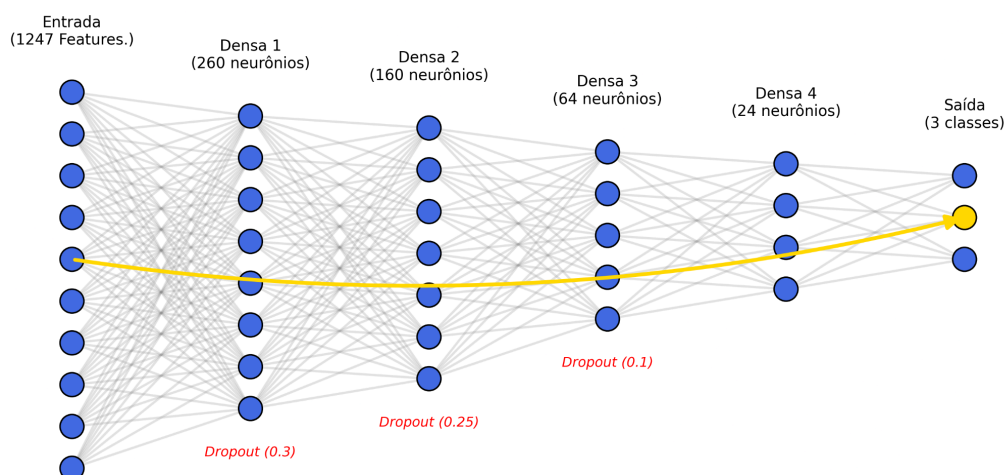


Fonte: Autores (2025).

### 5.3. Arquitetura da Rede Neural

Foi implementada uma Rede Neural Densa (DNN), também conhecida como *Perceptron* Multicamadas (MLP), cuja arquitetura é visualmente detalhada na Figura 4. A rede é projetada para receber um vetor de entrada contendo 1247 características extraídas dos sinais do acelerômetro. A estrutura consiste em quatro camadas ocultas densas sequenciais, com 260, 160, 64 e 24 neurônios, respectivamente, utilizando a função de ativação *ReLU* (*Rectified Linear Unit*) para o aprendizado de representações não-lineares. Para combater o *overfitting*, a técnica de regularização *Dropout* foi aplicada após as três primeiras camadas ocultas, com taxas decrescentes de 0.3, 0.25 e 0.1. Por fim, a camada de saída é composta por três neurônios com uma função de ativação *Softmax*, responsável por gerar a distribuição de probabilidade entre as três classes de atividade a serem classificadas ('andando', 'caindo' e 'sentando').

Figura 4 - Arquitetura do modelo neural desenvolvido.



Fonte: Autores (2025).

O treinamento da rede neural foi conduzido utilizando um conjunto de hiperparâmetros específicos, detalhados na Tabela 1, visando a otimização da performance do modelo. O processo de aprendizado foi executado por um total de 150 ciclos de treinamento (épocas), permitindo uma observação completa da evolução das curvas de perda e acurácia e a identificação do ponto ótimo de generalização. Para a atualização dos pesos do modelo, os dados de treinamento foram segmentados em lotes (*batches*) de 128 amostras por iteração, um tamanho que oferece um equilíbrio entre a eficiência computacional e a estabilidade na estimativa do gradiente. A magnitude dos ajustes nos pesos a cada atualização foi controlada por uma taxa de aprendizagem (*learning rate*) de 0.0005, um valor moderado que favorece uma convergência estável e gradual em direção ao mínimo da função de perda.

Tabela 1 - Hiperparâmetros

| Taxa de Aprendizagem<br>( <i>Learning Rate</i> ) | Tamanho do Lote<br>( <i>Batch Size</i> ) | Ciclos de Treinamento<br>( <i>Epochs</i> ) |
|--------------------------------------------------|------------------------------------------|--------------------------------------------|
| 0.0005                                           | 128                                      | 150                                        |

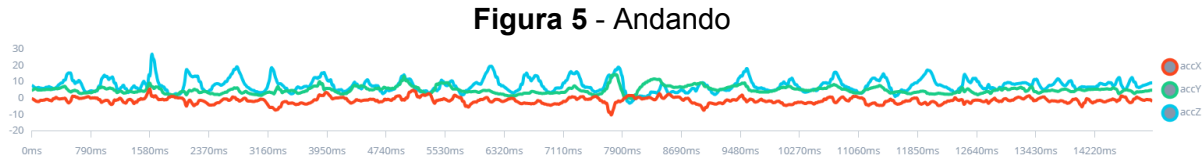
Fonte: Autores.

5.4.    **Análise descritiva dos dados**

A etapa de análise e processamento dos dados é fundamental para transformar os sinais brutos do acelerômetro em um formato otimizado para o treinamento da rede neural. Esta seção detalha a caracterização inicial dos sinais no domínio do tempo e, subsequentemente, o pipeline de Processamento Digital de Sinais (DSP) empregado para a extração de características.

5.4.1 **Caracterização dos Sinais no Domínio do Tempo**

A atividade Andando (Figura 5) é caracterizada por um padrão quasi-periódico e rítmico. As oscilações nos sinais, especialmente no eixo vertical, correspondem à cadência dos passos, exibindo uma amplitude moderada e uma estrutura cíclica que a distingue claramente de estados estáticos ou de eventos abruptos.



**Fonte:** Autores (2025).

O evento Caindo (Figura 6), por sua vez, apresenta uma assinatura não periódica e de alta energia, que é marcadamente distinta das outras classes. Tipicamente, o sinal de uma queda é composto por uma breve fase de instabilidade ou quase-ausência de gravidade (free-fall), seguida por um pico transiente de altíssima amplitude (alto valor de G), correspondente ao impacto do corpo com a superfície. Esta assinatura de impacto é um dos principais diferenciais para a classificação.

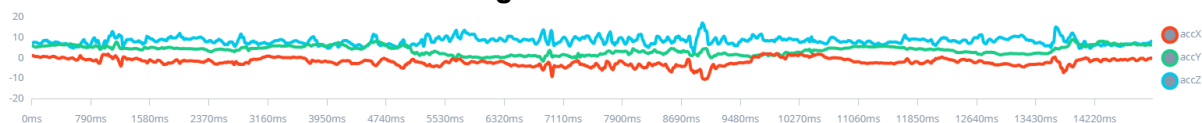
**Figura 6 - Caindo**



**Fonte:** Autores (2025).

A atividade Sentando (Figura 7) é um evento de baixa energia. O sinal é predominantemente estacionário, refletindo um estado de repouso, com uma pequena e breve variação de amplitude que denota o movimento suave de sentar-se. A baixa amplitude e a ausência de um padrão rítmico claro a diferenciam das demais atividades.

**Figura 7 - Sentando**



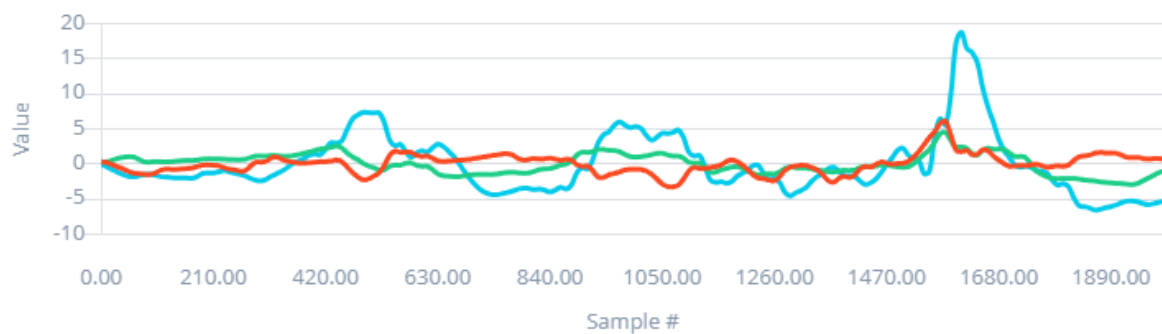
**Fonte:** Autores (2025).

#### 5.4.2 Caracterização dos Sinais no Domínio do Tempo

Para extrair características robustas dos sinais brutos, foi utilizado o pipeline de DSP da plataforma *Edge Impulse*. Este processo converte os dados do domínio do tempo para o domínio da frequência, onde os padrões de cada atividade se tornam mais evidentes. O pipeline é composto por dois blocos principais.

O primeiro bloco, *Filter*, prepara o sinal para a análise. Os parâmetros configurados foram *Scale axes*: 1 e *input decimation ratio*: 1, indicando que os dados do acelerômetro foram utilizados em sua escala original e sem decimação, preservando a taxa de amostragem completa para não perder informações de alta frequência contidas nos eventos de impacto. A Figura 8 ilustra o sinal após esta etapa inicial de processamento.

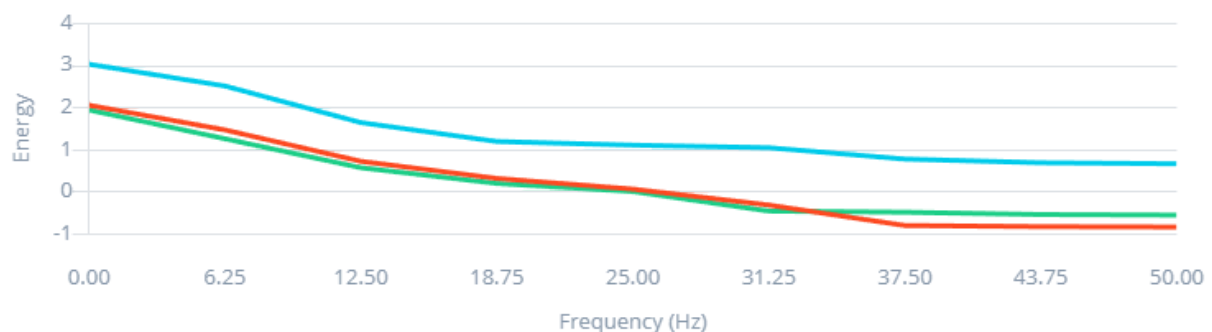
**Figura 8 - Sinal no domínio do tempo após o bloco de filtragem.**



**Fonte:** Autores (2025).

O resultado final deste pipeline de *DSP* é um espectrograma, como representado na Figura 9. O espectrograma é uma imagem que mostra o tempo no eixo X, a frequência no eixo Y e a intensidade (potência) de cada frequência em cada instante de tempo como uma cor. É este espectrograma que é então "achatado" para formar o vetor de características que alimenta a rede neural, contendo informações ricas sobre a distribuição de energia espectral do sinal ao longo do tempo.

**Figura 9** - Espectrograma (potência espectral em escala logarítmica) gerado pelo bloco de análise *FFT*.



**Fonte:** Autores (2025).

## 6. Resultados

O modelo foi avaliado no conjunto de validação, que não foi utilizado durante o treinamento. A performance do modelo final, quantizado em *int8* para otimização em dispositivos embarcados, foi avaliada por um conjunto abrangente de métricas, conforme consolidado na Tabela 2. O modelo alcançou uma acurácia geral de 88.1%, indicando uma alta taxa de acerto nas classificações totais. A robustez do classificador é corroborada por uma excelente Área sob a Curva *ROC* (*AUC*) de 0.96, o que demonstra uma capacidade discriminativa superior entre as diferentes classes de atividade. Adicionalmente, as métricas ponderadas de desempenho, que consideram o desbalanceamento entre as classes, revelam um comportamento equilibrado e robusto. Com uma precisão ponderada de 0.89, o modelo minimiza a ocorrência de falsos alarmes, enquanto um recall ponderado de 0.88 garante a detecção da vasta maioria dos eventos reais. O *F1-score* ponderado de 0.88 sintetiza este equilíbrio, confirmando que o modelo mantém alta performance tanto na identificação correta de eventos quanto na prevenção de classificações errôneas, validando sua eficácia para a aplicação proposta.

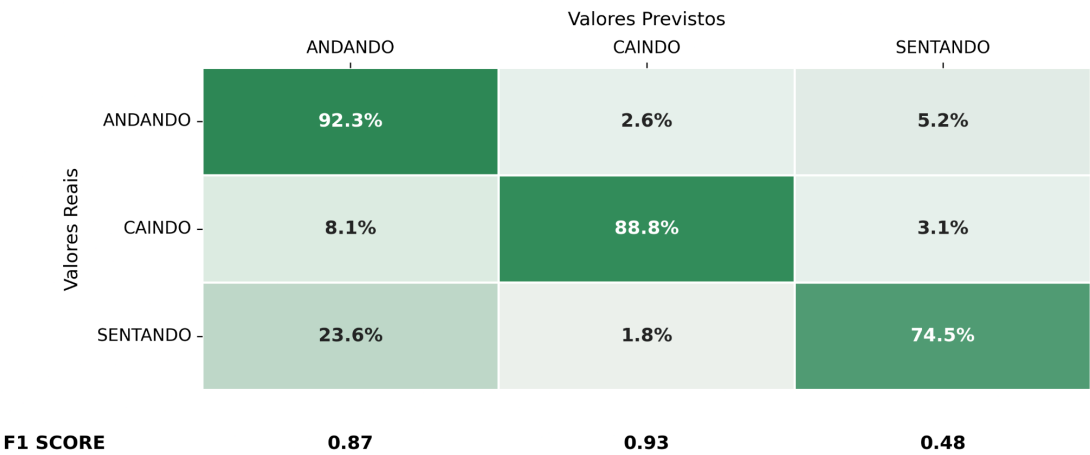
Tabela 2 - Métricas de desempenho obtido.

| Acurácia<br>(Accuracy) | Perda<br>(Loss) | Área sob a<br>Curva ROC<br>(AUC) | Métricas<br>Ponderadas<br>(Weighted<br>average<br>Precision) | Weighted<br>average<br>Recall | Weighted<br>average F1<br>score |
|------------------------|-----------------|----------------------------------|--------------------------------------------------------------|-------------------------------|---------------------------------|
| 88.1%                  | 0.44            | 0.96                             | 0.89                                                         | 0.88                          | 0.88                            |

Fonte: Autores (2025).

A matriz de confusão, apresentada na Figura 10, oferece um panorama detalhado do desempenho por classe.

Figura 10: Matriz de Confusão do Modelo de Validação (int8)  
Confusion Matrix



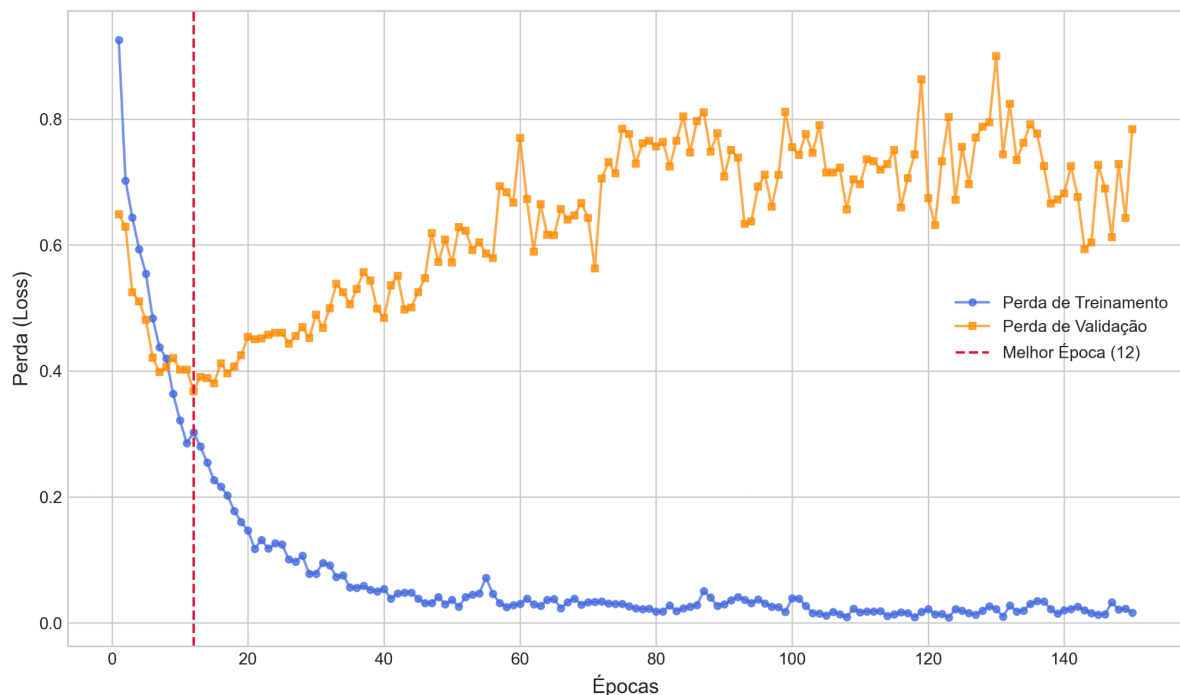
Fonte: Autores (2025).

A análise da dinâmica de treinamento, ilustrada no Gráfico de Perda (Figura 11), revela informações cruciais sobre o comportamento de aprendizado do modelo. Observa-se que a perda de treinamento (em azul) decresce de forma consistente ao longo das 150 épocas, estabilizando-se em um valor próximo a zero, o que indica que a arquitetura proposta possui capacidade suficiente para aprender os padrões do conjunto de dados de treino.

Contudo, a curva de perda de validação (em laranja), que mede a capacidade de generalização do modelo em dados não vistos, exibe um comportamento distinto. Ela atinge seu valor mínimo na 12ª época e, subsequentemente, inicia uma tendência de ascensão clara e errática. A divergência acentuada entre as curvas de treinamento e validação a partir deste ponto é um indicador clássico do fenômeno de *overfitting*. Isso sugere que, após a 12ª

época, o modelo começa a memorizar o ruído e as particularidades dos dados de treinamento em detrimento da aprendizagem de características generalizáveis, tornando-se menos eficaz para novas amostras. Portanto, o modelo com o melhor desempenho preditivo é aquele obtido na 12ª época, ponto que representa o balanço ótimo entre viés e variância para este problema.

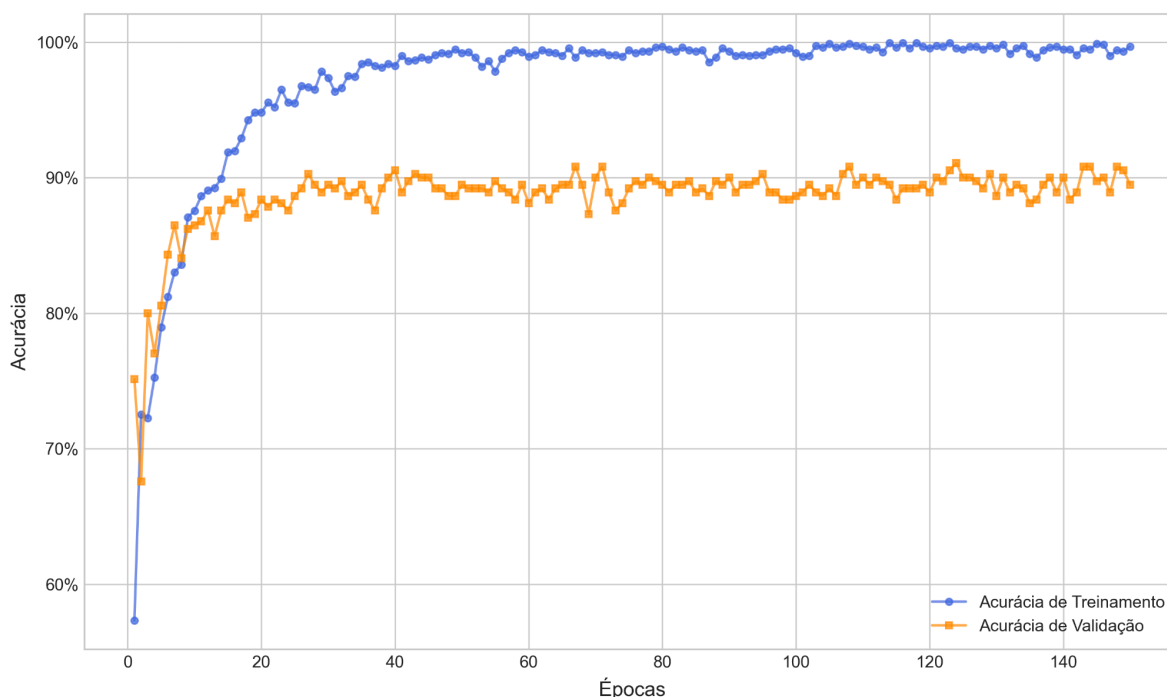
**Figura 11** - Histórico de Perda Durante o Treinamento.



**Fonte:** Autores (2025).

Complementarmente à análise da função de perda, o Gráfico de Acurácia (Figura 12) oferece uma perspectiva sobre a taxa de acerto do modelo. A curva de acurácia de treinamento (em azul) demonstra uma rápida convergência, aproximando-se de 100% de acerto, o que reitera a alta capacidade do modelo de se ajustar aos dados de treinamento. Em contrapartida, a acurácia de validação (em laranja), após um crescimento inicial similar, entra em um estado de saturação de desempenho, estabilizando-se em um platô por volta de 90% a partir da 30ª época, aproximadamente. O hiato significativo e persistente que se estabelece entre as duas curvas é uma evidência visual que corrobora o diagnóstico de *overfitting*. Isso indica que, embora o modelo memorize os dados de treino com perfeição, sua capacidade de generalizar e classificar corretamente novas amostras atinge um limite, estabelecendo a performance efetiva do modelo em torno de 90%.

**Figura 12 - Histórico de Acurácia Durante o Treinamento**



**Fonte:** Autores (2025).

## 7. Discussão

Os resultados demonstram a viabilidade da abordagem, porém destacam limitações associadas ao desequilíbrio de classes, especialmente para o estado “sentando”. A acurácia geral de 87.6% e a alta *AUC* de 0.96 são indicadores fortes de que o modelo aprendeu a distinguir padrões relevantes nos sinais de vibração. A classe 'caindo' obteve o melhor desempenho individual (Precisão de 97.2% e *Recall* de 88.1%), o que é extremamente positivo, dado que a detecção de quedas é o requisito mais crítico do sistema.

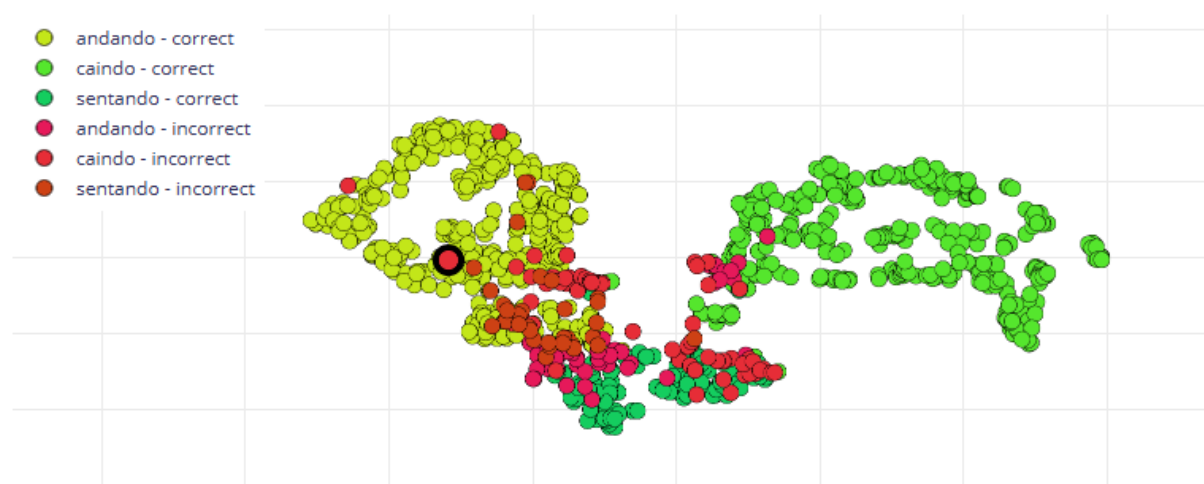
A análise da matriz de confusão revela pontos de melhoria. A classe 'sentando' foi a mais desafiadora, com a menor precisão (71.9%). Notavelmente, 14 amostras de 'sentando' foram erroneamente classificadas como 'andando'. Esta confusão pode ser atribuída a dois fatores principais:

1. Desbalanceamento de Dados: A classe 'sentando' possuía apenas 4 amostras para treinamento, em comparação com 12 para 'andando' e 'caindo'. Essa sub-representação dificulta o aprendizado de suas características distintivas pela rede.
2. Similaridade de Sinais: O ato de sentar pode conter transições de movimento que se assemelham, em certos aspectos, ao início ou fim de uma caminhada, confundindo o classificador.

Há também uma confusão bidirecional entre 'andando' e 'caindo' (12 erros de 'caindo' para 'andando' e 4 do inverso), sugerindo que as fases de transição desses movimentos possuem sobreposição de características.

Na amostra *caindo.unknown.61aovh5s.ingestion-78468cb6bb-bhdqn*, o modelo classificou erroneamente, observe na Figura X. O algoritmo classificou a vibração como status “andando”, sendo que na verdade está caindo.

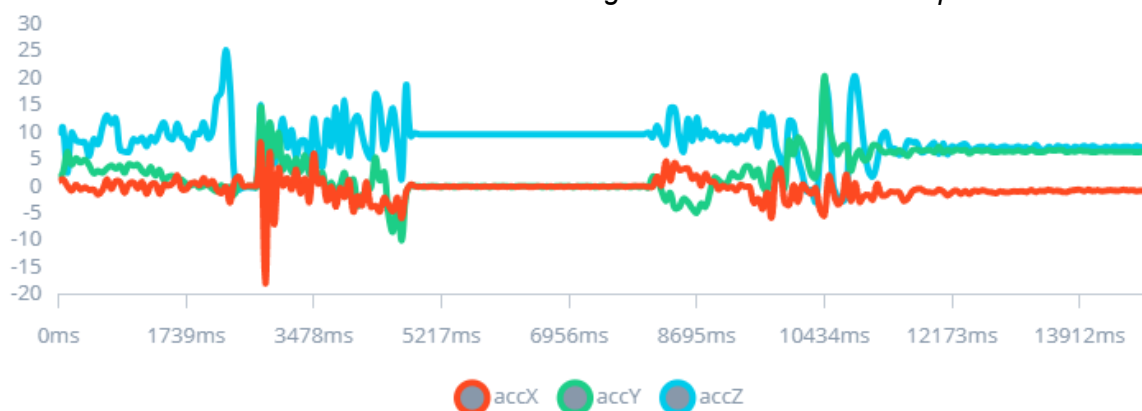
**Figura 13** - Classificação incorreta da amostra *caindo.unknown.61aovh5s.ingestion-78468cb6bb-bhdqn*



**Fonte:** Autores (2025).

Essa classificação errônea se deve ao fato de que, na Figura 14, há uma acentuada variação de vibração e logo em seguida uma variação simulando uma pessoa se levantando lentamente. O que fez o modelo compreender como se estivesse andando normalmente.

**Figura 14** - Variações das vibrações dos eixos ao longo do tempo da amostra *caindo.unknown.61aovh5s.ingestion-78468cb6bb-bhdqn*.



**Fonte:** Autores (2025).



## 8. Conclusão

Este estudo validou com sucesso a aplicação de um sistema de baixo custo, baseado em smartphone e na plataforma Edge Impulse, para o monitoramento de atividades de idosos. O modelo de rede neural densa proposto atingiu uma acurácia de validação de 88.1% e uma AUC de 0.96, demonstrando alta capacidade para distinguir atividades críticas como quedas, o que estabelece a viabilidade da abordagem. A principal contribuição do trabalho é a demonstração de um fluxo de desenvolvimento acessível para a criação de tecnologias assistivas. As limitações identificadas, como o overfitting e a inadequação da arquitetura MLP para dados temporais, definem as diretrizes para trabalhos futuros, que incluirão a adoção de Redes Neurais Convolucionais 1D (1D-CNN), o enriquecimento do dataset com mais amostras e o uso de técnicas de aumento de dados (data augmentation) para aprimorar a robustez e a capacidade de generalização do modelo.

## Referências

RIBEIRO, Nathan Faustino. Previsão de velocidade do vento com dados reais de um parque eólico localizado no Ceará utilizando redes neurais artificiais: um estudo comparativo das redes MLP, LSTM, GRU e CNN. 2024.

AQUINO, Gustavo de Aquino. Explicações visuais aplicadas a redes neurais convolucionais unidimensionais com ênfase no reconhecimento humano de atividades. 2024.

CLASSIFICAÇÃO de atividades humanas em tempo real para monitoramento de idosos utilizando redes neurais embarcadas e sinais de acelerômetro. [S. l.], 2023.

WANG, J.; CHEN, Y.; HAO, S.; PENG, X.; HU, L. Deep learning for sensor-based activity recognition: A survey. *Pattern Recognition Letters*, v. 119, p. 3-11, 2019.

WORLD HEALTH ORGANIZATION. Falls. Fact sheet. Genebra: WHO, 2021.

CTRNN FRAMEWORK. *A trainable low-power continuous-time recurrent neural network framework for real-time human activity recognition in embedded healthcare applications*. *Applied Sciences*, v. 15, n. 13, p. 7508, 2022.

GRU + ATTENTION. *Gate Recurrent Units with channel and temporal attentions for efficient sensor-based HAR on embedded devices*. *Electronics*, v. 11, n. 11, p. 1797, 2022.

SENSOR-EMBEDDED CNNs. *Sensor-embedded convolutional neural networks on low-cost microcontrollers for real-time HAR*. *Sensors*, v. 23, n. 19, p. 8127, 2023.

HYBRID CNN-GRU. *A Hybrid Deep Neural Network for Human Activity Recognition*. *International Journal of Advanced Computer Science and Applications*, v. 12, n. 11, p. 273-283, 2021.

3D-CNN HAR. *A deep learning method based on 3D convolutional neural networks for human activity recognition*. *Sensors*, v. 23, n. 5, p. 2816, 2023.

## Anexo 1 - Matriz de Confusão

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# --- 1A. DADOS NUMÉRICOS (para as cores do heatmap) ---
# Primeiro, definimos os dados como números puros.
data_numeric = {
    'ANDANDO': [92.3, 8.1, 23.6],
    'CAINDO': [2.6, 88.8, 1.8],
    'SENTANDO': [5.2, 3.1, 74.5],
}

# Rótulos para as linhas (Valores Reais)
index_labels = ['ANDANDO', 'CAINDO', 'SENTANDO']

# Criar o DataFrame numérico
df_conf_matrix_numeric = pd.DataFrame(data_numeric, index=index_labels)

# --- 1B. DADOS EM TEXTO (para as anotações nas células) ---
# Agora, criamos um segundo DataFrame para as anotações,
# formatando cada número do DataFrame anterior como uma string com '%'.
df_conf_matrix_annot = df_conf_matrix_numeric.applymap(lambda x: f'{x:.1f}%')

# Dados do F1 Score (sem alteração)
f1_scores = {
    'ANDANDO': [0.87],
    'CAINDO': [0.93],
    'SENTANDO': [0.48]
}
df_f1 = pd.DataFrame(f1_scores, index=['F1 SCORE'])

# --- 2. Criação da Visualização ---
fig, ax = plt.subplots(figsize=(10, 5))
fig.suptitle('Confusion Matrix', fontsize=16, y=1.02, x=0.45)
cmap_custom = sns.light_palette("seagreen", as_cmap=True)

# --- 3. Plotando o Heatmap da Matriz de Confusão ---
# ATENÇÃO ÀS MUDANÇAS AQUI:
sns.heatmap(df_conf_matrix_numeric,          # 1. Usamos os dados NUMÉRICOS para as
cores                                         cores
            annot=df_conf_matrix_annot,      # 2. Usamos o DataFrame de TEXTO para as
anotações                                    anotações
            fmt="",                          # 3. REMOVEMOS o 'fmt' pois a anotação já está
formatada                                    formatada
            cmap=cmap_custom,
            linewidths=1,
```

```

        linecolor='white',
        cbar=False,
        ax=ax,
        annot_kws={"size": 12, "weight": "bold"})

# Customização dos rótulos e eixos (sem alteração)
ax.set_xlabel('Valores Previstos', fontsize=12, labelpad=10)
ax.set_ylabel('Valores Reais', fontsize=12, labelpad=10)
ax.xaxis.set_ticks_position('top')
ax.xaxis.set_label_position('top')
ax.tick_params(axis='x', labelsiz=11, rotation=0)
ax.tick_params(axis='y', labelsiz=11, rotation=0)

# --- 4. Adicionando a Linha do F1 Score (sem alteração) ---
for i, col in enumerate(df_f1.columns):
    f1_value = df_f1[col].values[0]
    if isinstance(f1_value, (int, float)):
        text_to_display = f'{f1_value:.2f}'
    else:
        text_to_display = ""
    ax.text(i + 0.5, 3.5, text_to_display, ha='center', va='center', fontsize=12, weight='bold')

ax.text(-0.5, 3.5, 'F1 SCORE', ha='center', va='center', fontsize=12, weight='bold')

# --- 5. Ajustes Finais e Salvamento ---
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('matriz_confusao_modelo_percent.png', dpi=300, bbox_inches='tight')
print("Imagem 'matriz_confusao_modelo_percent.png' salva com sucesso!")
plt.show()

```

**Fonte:** Autores (2025).